

KevaBook Database Design Document (DBDD)

1. Introduction

1.1 Purpose

This Database Design Document (DBDD) defines the logical and physical database design for the KevaBook platform. It describes how application data is organized, stored, constrained, and managed across the platform's microservices.

The document serves as the primary reference for implementing the database layer and complements the Software Requirements Specification (SRS), Domain Model, Business Rules, and Software Architecture Document (SAD).

The objectives of this document are to:

- Define the database architecture adopted by the KevaBook platform.
- Describe the database schema for each microservice.
- Specify tables, columns, primary keys, foreign keys, constraints, and indexes.
- Document data ownership and relationships within each service.
- Define database naming conventions and design standards.
- Describe strategies for database migration, backup, recovery, and future scalability.

This document is intended to ensure consistency, maintainability, and data integrity throughout the implementation and evolution of the KevaBook platform.

1.2 Scope

This document covers the database design for all microservices that comprise the KevaBook platform. The current scope includes the following service databases:

- User Service Database
- Business Service Database
- Service Catalog Service Database
- Booking Service Database
- Notification Service Database

Each microservice owns its database and is solely responsible for creating, reading, updating, and deleting its data. No database is shared between services, and direct cross-service database access is prohibited.

The document includes:

- Database architecture
- Database design principles
- Schema definitions
- Table structures
- Relationships within each database
- Constraints
- Indexing strategy
- Data integrity rules
- Database security considerations
- Migration strategy
- Backup and recovery strategy

Business rules and domain concepts are defined in separate documents and are referenced where applicable.

1.3 Intended Audience

This document is intended for:

- Software Architects
- Backend Developers
- Database Engineers
- DevOps Engineers
- System Integrators
- Quality Assurance Engineers
- Technical Reviewers

It provides the technical information required to implement, maintain, test, and evolve the KevaBook database layer.

1.4 References

This Database Design Document should be read in conjunction with the following project documentation:

- KevaBook Software Requirements Specification (SRS)
- KevaBook Domain Model
- KevaBook Business Rules
- KevaBook Software Architecture Document (SAD)

Together, these documents provide a complete specification of the KevaBook platform from business requirements through implementation.

2. Database Design Overview

This section presents the overall database design strategy adopted for the KevaBook platform. It outlines the architectural principles, database technologies, and data management practices that guide the design of all service databases. The KevaBook platform follows a Database-per-Service architecture, where each microservice owns and manages its own independent PostgreSQL database. This approach supports service autonomy, scalability, maintainability, and clear ownership of data while preventing tight coupling between services.

2.1 Database Management System

KevaBook uses **PostgreSQL** as the relational database management system (RDBMS) for all microservices.

PostgreSQL was selected because it provides:

- Full ACID (Atomicity, Consistency, Isolation, Durability) transaction support.
- Strong referential integrity through primary and foreign key constraints.
- High reliability for transactional workloads.
- Excellent performance for complex queries.
- Advanced indexing capabilities.
- Native support for JSON data types when semi-structured data is required.
- Mature tooling, extensive documentation, and broad community support.

These characteristics make PostgreSQL well suited for a booking platform where transactional consistency and data integrity are essential.

2.2 Database-per-Service Architecture

KevaBook adopts the Database-per-Service architectural pattern. Each microservice owns its own database and is solely responsible for managing its data. Other microservices are prohibited from directly accessing another service's database. Instead, services communicate through:

- HTTP REST interfaces for synchronous operations.

- RabbitMQ domain events for asynchronous communication.

This architecture provides several benefits:

- Clear data ownership.
- Independent schema evolution.
- Loose coupling between services.
- Independent deployment of database changes.
- Improved fault isolation.
- Enhanced scalability.

Cross-service data sharing is achieved through service APIs or domain events rather than direct database queries.

2.3 Data Ownership

Each database is owned exclusively by its corresponding microservice. The ownership model is summarized below

Microservice	Database Ownership
User Service	Business Owners and user account information
Business Service	Businesses, Staff, Locations, and Business Settings
Service Catalog Service	Services offered by Businesses
Booking Service	Bookings and Client booking information
Notification Service	Notification records and delivery status

Each microservice is responsible for maintaining the integrity, consistency, and lifecycle of the data stored within its database.

2.4 Transaction Management

Transactions are managed independently within each microservice. Database transactions are limited to the boundaries of a single service database. Distributed transactions spanning multiple databases are intentionally avoided. Where business operations involve multiple services, consistency is maintained through asynchronous messaging using RabbitMQ and the principles of eventual consistency.

This approach aligns with modern microservices architecture by promoting service independence while maintaining reliable business workflows.

2.5 Database Normalization

The KevaBook databases are designed following the principles of Third Normal Form (3NF). Normalization minimizes data redundancy, improves data integrity, and simplifies database maintenance.

The design aims to:

- Eliminate duplicate data.
- Ensure that each table represents a single business concept.
- Maintain clear relationships between related entities.
- Reduce update, insert, and delete anomalies.

Where justified by performance requirements, controlled denormalization may be considered in future versions of the platform.

2.6 Primary Key Strategy

The KevaBook platform uses Long values as the primary key for all database entities.

Each microservice generates and manages its own identifiers within its dedicated PostgreSQL database. Since every service owns an independent database, primary key uniqueness is required only within the scope of that service.

This strategy provides several advantages:

- Efficient storage compared to universally unique identifiers (UUIDs).
- Faster indexing and query performance.
- Smaller index sizes, improving database efficiency.
- Simpler database administration and debugging.
- Excellent compatibility with PostgreSQL auto-generated identity columns.

Primary keys are immutable and uniquely identify each record within their respective service database.

Cross-service relationships are maintained using identifier references exchanged through HTTP REST APIs and domain events rather than database-level foreign key constraints.

2.7 Foreign Key Strategy

Foreign key relationships are enforced only within the boundaries of a single service database.

Relationships between entities owned by different microservices are maintained using identifier references rather than database-level foreign key constraints.

For example:

- The Booking Service stores `businessId`, `staffId`, and `serviceId` as identifier references.
- Validation of these references is performed through HTTP REST communication with the owning microservices rather than through direct database relationships.

This strategy preserves service autonomy and prevents tight coupling between databases.

2.8 Audit Information

To support traceability and operational monitoring, business entities include standard audit attributes where applicable.

Typical audit fields include:

- `createdAt`
- `updatedAt`

These fields record when a record was created and last modified, supporting auditing, troubleshooting, and reporting.

Future versions of the platform may extend audit capabilities with additional attributes such as:

- `createdBy`
- `updatedBy`
- Soft deletion metadata (`deletedAt`, `deletedBy`)

2.9 Database Design Objectives

The KevaBook database design is guided by the following objectives:

- Ensure strong data integrity.
- Support transactional consistency.
- Promote service autonomy.

- Enable independent schema evolution.
- Eliminate unnecessary data duplication.
- Support efficient querying and indexing.
- Facilitate future scalability.
- Maintain strict multi-tenant data isolation.
- Simplify long-term maintenance and evolution.

These objectives ensure that the database layer supports the functional and non-functional requirements of the KevaBook platform while remaining adaptable to future business growth.

3. Database Architecture

This section describes the overall database architecture adopted by the KevaBook platform. The architecture follows the Database-per-Service pattern, where each microservice owns and manages its own dedicated PostgreSQL database.

This approach ensures clear data ownership, service autonomy, independent schema evolution, and strong separation of concerns. Each database is accessed exclusively by its owning microservice, and direct database access between services is strictly prohibited.

Microservices exchange information through HTTP REST interfaces for synchronous operations and RabbitMQ domain events for asynchronous communication.

3.1 Database-per-Service Architecture

KevaBook adopts the Database-per-Service architectural pattern to support a scalable and loosely coupled microservices architecture. Each microservice owns its own PostgreSQL database and is solely responsible for managing the lifecycle of its data.

The platform currently consists of the following service databases:

Microservice	Database	Primary Responsibility
User service	userDb	Business owner accounts and user management
Business service	businessDb	Business, staff, Locations and business settings

Service Catalog Service	servicecatalogDb	Business services offered to clients
Booking service	bookingDb	Booking management and client booking information
Notification service	notificationDb	Notification history and delivery tracking

Each database operates independently and can evolve without impacting the schemas of other services.

3.2 Database Ownership

Each database is owned exclusively by a single microservice.

Ownership includes responsibility for:

- Creating and maintaining database schemas.
- Managing database migrations.
- Enforcing data integrity.
- Performing Create, Read, Update, and Delete (CRUD) operations.
- Managing indexes and constraints.
- Maintaining transactional consistency.

No other microservice is permitted to access another service's database directly.

3.3 Cross-Service Data References

Business entities frequently reference data owned by other services. Rather than using cross-database foreign keys, KevaBook stores identifier references. For example:

- The Booking Service stores:
 - businessId
 - staffId
 - serviceId

These identifiers reference entities owned by the Business Service and Service Catalog Service. Validation of these references is performed through HTTP REST communication with the owning microservice before data is persisted. This approach preserves service independence while maintaining business consistency.

3.4 Database Communication

Each database communicates exclusively with its owning microservice.

Communication between microservices occurs through:

- HTTP REST interfaces for synchronous operations.
- RabbitMQ for asynchronous event-driven communication.

Databases never communicate directly with one another.

This architecture eliminates tight coupling between persistence layers and reinforces the principle of independent service ownership.

3.5 Database Isolation

To ensure security, reliability, and tenant isolation, each PostgreSQL database operates independently.

The architecture enforces the following principles:

- Every database belongs to exactly one microservice.
- Database credentials are unique for each service.
- Direct cross-database queries are prohibited.
- Each service manages its own transactions.

This isolation reduces the impact of failures and enables independent deployment and maintenance of each service.

3.6 Database Architecture Principles

The KevaBook database architecture is guided by the following principles:

- Database ownership resides with a single microservice.
- No shared databases between services.
- No direct cross-service database access.
- Independent schema evolution.
- Independent database deployment.
- Strong transactional consistency within each service.
- Eventual consistency across multiple services.
- Service communication through APIs and domain events.
- Clear separation of data responsibilities.
- Scalability through independent database management.

These principles establish a robust and maintainable data architecture capable of supporting the long-term evolution of the KevaBook platform.

4. Database Schema Design

This section presents the detailed database schema for each microservice within the KevaBook platform. In accordance with the Database-per-Service architectural pattern, every microservice owns an independent PostgreSQL database containing only the tables required to support its business responsibilities.

Each schema has been designed to:

- Maintain strong data integrity.
- Support transactional consistency.
- Eliminate unnecessary data redundancy.
- Clearly define ownership of business data.
- Support efficient querying through appropriate indexing.
- Enable independent schema evolution.

The following subsections describe the database schema for each microservice.

4.1 User Service Database

Purpose

The User Service Database stores information related to Business Owner accounts. It is responsible for maintaining user profile information and linking authenticated users to the businesses they own or manage. Authentication and credential management are delegated to Keycloak. Consequently, this database stores only application-specific user information and the reference to the corresponding Keycloak account.

Database Name

userDb

Database Responsibilities

The User Service Database is responsible for:

- Managing Business Owner information.
- Linking Business Owners to their Business.
- Storing the external Keycloak user identifier.
- Managing application-specific user metadata.

The database is **not responsible** for:

- Authentication.
- Password storage.
- Authorization policy management.
- User session management.

These responsibilities are delegated to Keycloak.

Relationships

Internal relationships:

- One Business Owner belongs to one Business.

External references:

- `businessId` references a Business managed by the Business Service.

This reference is maintained through inter-service communication rather than a database-level foreign key.

Design Considerations

The User Service Database follows the principle of minimal data ownership by storing only information required for application functionality. Identity management remains centralized within Keycloak, avoiding duplication of authentication data and simplifying user management across the platform.

4.1.1 Table Design – `businessOwners`

Purpose

The `businessOwners` table stores application-specific information about Business Owners registered on the KevaBook platform. It links authenticated users from Keycloak to the businesses they own or manage.

Authentication credentials, passwords, and user sessions are **not stored** in this table. These responsibilities are handled by Keycloak.

Table Structure

Column	Data Types	Constraints	Description
ownerId	Long	Primary Key	Unique identifier of business owner
businessId	Long	Not null, unique	Reference the business table
firstName	String	Not Null	First name
lastName	String	Not Null	Last name
email	String	Not null	Work or personal email
phoneNumber	String	Null	Contact phone number
role	Enum	Not Null	Application role
createdAt	Timestamp	Not Null	Creation date time
updatedAt	Timestamp	Not Null	Update date time

Foreign Key Strategy

The businessId column represents an application-level reference to a Business managed by the Business Service. Because KevaBook follows the **Database-per-Service** architectural pattern, no database-level foreign key constraint is defined between the User Service Database and the Business Service Database. Validation of the businessId is performed through inter-service communication.

4.2 Business Service Database

Purpose

The Business Service Database stores the core business information for the KevaBook platform. It manages Business entities and their associated operational data, including Staff, Locations, and Business Settings.

Each Business represents an independent tenant within the platform. The database is responsible for maintaining tenant-specific information while enforcing strict data ownership and isolation.

This database serves as the authoritative source for business-related information used throughout the platform.

Database Name

businessDb

Tables

Table	Purpose
businesses	Stores information about each registered Business (tenant).
staff	Stores Staff members associated with a Business.
locations	Stores physical or virtual service locations.
business_settings	Stores configurable settings specific to each Business.

Database Responsibilities

The Business Service Database is responsible for:

- Managing Business information.
- Managing Staff members.
- Managing Business Locations.
- Managing Business configuration and operational settings.
- Enforcing tenant ownership within the Business domain.

The database is **not responsible** for:

- Managing authentication or user credentials.
- Managing bookable Services.
- Managing Bookings.
- Managing Notifications.

These responsibilities belong to their respective microservices.

Relationships

The Business Service Database contains the following internal relationships:

- One Business has many Staff members.
- One Business has many Locations.
- One Business has one Business Settings record.

These relationships are enforced using database-level foreign key constraints because they exist within the same service database.

External References

Other microservices reference Business data through identifier values rather than direct database relationships.

Examples include:

- The Booking Service references `business_id` and `staff_id`.
- The User Service references `business_id`.
- The Service Catalog Service references `business_id`.

Validation of these references is performed through inter-service communication using HTTP REST APIs.

Design Considerations

The Business Service Database is designed as the tenant management database for the KevaBook platform.

Its primary objectives are to:

- Maintain Business ownership and identity.
- Support multiple Staff members per Business.
- Support multiple service Locations.
- Maintain configurable Business settings.
- Provide a stable foundation for other microservices through identifier references.

The schema is normalized to Third Normal Form (3NF) to reduce redundancy, maintain consistency, and simplify future schema evolution.

4.2.1 Table Design – businesses

Purpose

The businesses table stores information about every Business registered on the KevaBook platform. Each record represents an independent tenant and serves as the primary business entity within the Business Service.

The table maintains the core business profile and provides the tenant identifier (business_id) that is referenced by other microservices through application-level relationships.

Table Structure

Column	Data Type	Constraints	Description
businessId	Long	Primary key	Unique identifier for the Business
name	String	Not Null	Registered business name.
description	String	Null	Description of the business and the services it provides
email	String	Not Null, Unique	Primary business contact email
phoneNumber	String	Not Null	Primary business number
status	Enum	Not Null	Current business status
createdAt	Timestamp	Not Null	Creation date time
updatedAt	Timestamp	Not Null	Updated date time

Relationships

Relationship	Type	Description
Business and Staff	One-to-Many	A Business can have multiple Staff members.
Business and Locations	One-to-Many	A Business can have multiple Locations.
Business and Business Settings	One-to-One	A Business has a single Business Settings record.

Design Notes

- Each Business represents a single tenant within the KevaBook platform.
- The business_id serves as the tenant identifier throughout the platform.
- Other microservices reference the Business using business_id; however, no cross-database foreign key constraints are defined.
- Business lifecycle management (e.g., activation and suspension) is controlled through the status column.
- Audit fields (created_at and updated_at) support traceability and operational monitoring.

4.2.2 Table Design – staff

Purpose

The staff table stores information about individuals associated with a Business who are responsible for delivering bookable services. Each Staff member belongs to exactly one Business and may be assigned one or more Bookings based on their Availability.

The table maintains staff profile information required for business operations and scheduling.

Table Structure

Column	Data Type	Constraints	Description
staffId	Long	Primary Key	Unique identifier for staff member
businessId	Long	Not Null, Reference key	Reference to business to which staff member belongs
firstName	String	Not Null	First name
lastName	String	Not Null	Last name
phoneNumber	String	Not Null	Phone number

email	String	Unique, Not Null	Email
role	Enum	Not Null	Business role or speciality
active	boolean	Not Null	Indicates whether the Staff member is currently active.
createdAt	Timestamp	Not Null	Creation date time
updatedAt	Timestamp	Not Null	Updated date time

Relationships

Relationship	Type	Description
Staff and Business	Many-to-One	Each Staff member belongs to one Business.
Staff and Availability	One-to-Many	A Staff member may have multiple Availability schedules.
Staff and Bookings	One-to-Many	A Staff member may be associated with multiple Bookings stored in the Booking Service.

The relationship between Staff and Bookings is maintained through identifier references and service communication, as Bookings are stored in a separate database.

Design Notes

- Every Staff member belongs to exactly one Business.
- The business_id enforces tenant ownership within the Business Service database.
- Staff profile information is maintained independently of authentication and authorization.
- Availability schedules are maintained separately to support flexible working patterns.
- Booking assignments are managed by the Booking Service using staff_id references.
- Soft deletion is not used; Staff availability is controlled through the active flag.

4.2.3 Table Design – locations

Purpose

The locations table stores the physical or virtual locations where a Business provides its services. Each Location belongs to exactly one Business and represents a place where clients can receive booked services.

Supporting multiple Locations enables Businesses to operate from different branches, offices, studios, or provide virtual services through online meeting platforms.

Table Structure

Colum	Data Type	Constraints	Description
locationId	Long	Primary Key	Unique identifier
businessId	Long	Not Null	References the Business that owns the Location
street	String	Not Null	Street name of the Location
city	String	Not Null	City where the Location
postalCode	String	Null	Postal code
country	String	Not Null	Country
LocationType	Enum	Not Null	Type of Location
createdAt	Timestamp	Not Null	Creation date time
updatedAt	Timestamp	Not Null	Updated date time

Relationships

Relationship	Type	Description
--------------	------	-------------

Location and Business	Many-to-One	Each Location belongs to exactly one Business.
-----------------------	-------------	--

Design Notes

- Every Location belongs to exactly one Business.
- A Business may operate from one or more Locations.
- Locations may represent either physical premises or virtual service environments.
- The business_id foreign key enforces tenant ownership within the Business Service database.

4.3 Service Catalog Service Database

Purpose

The Service Catalog Service Database manages the catalog of services offered by Businesses on the KevaBook platform. It stores information about each bookable service, including its duration, pricing, availability status, and the Staff members qualified to perform the service.

The database acts as the authoritative source for all service-related information and supports flexible assignment of Services to multiple Staff members.

Tables

Table	Purpose
Services	Stores all bookable services offered by a business
StaffService	Associates Staff members with the Services they are qualified to perform

Database Responsibilities

The Service Catalog Service Database is responsible for:

- Managing the lifecycle of Services.
- Managing service pricing and duration.
- Managing service availability.
- Assigning Services to qualified Staff members.

- Providing service information to the Booking Service during booking validation.

The database is **not responsible** for:

- Managing Businesses.
- Managing Staff profiles.
- Managing Bookings.
- Managing Notifications.

These responsibilities belong to their respective microservices.

Relationships

The Service Catalog Service Database contains the following internal relationship:

- One Service may be assigned to many Staff members.
- One Staff member may perform many Services.

This many-to-many relationship is implemented using the `staff_services` junction table.

External References

The Service Catalog Service stores identifier references to entities owned by the Business Service.

Examples include:

- `business_id` references a Business.
- `staff_id` references a Staff member.

These references are maintained through application-level validation using HTTP REST APIs rather than database-level foreign key constraints.

Design Considerations

The Service Catalog Service Database is designed to support flexible service management while maintaining loose coupling between microservices. The use of a junction table for Staff-to-Service assignments ensures that Businesses can accurately model which Staff members are qualified to deliver specific Services. This approach supports real-world scenarios where not every Staff member performs every available Service.

4.3.1 Table Design – services

Purpose

The services table stores the bookable services offered by Businesses on the KevaBook platform. Each Service belongs to exactly one Business and defines the information required for clients to make bookings, including service duration, pricing, and availability status.

The table serves as the authoritative source for service definitions used throughout the platform.

Table Structure

Column	Data Type	Constraints	Description
service_id	Long	Primary Key	Unique identifier for the Service.
business_id	Long	Not Null	Identifier of the Business offering the Service. This is an application-level reference to the Business Service.
name	String	Not Null	Name of the Service.
description	String	Null	Detailed description of the Service.
duration_minutes	Integer	Not Null	Standard duration of the Service in minutes.
price	Decimal	Not Null	Standard price charged for the Service.
currency	String	Not Null	Currency code
status	String	Not Null	Current status of the Service
created_at	Timestamp	Not Null	Date Service was created.

updated_at	Timestamp	Not Null	Date and time the Service was last updated.
------------	-----------	----------	---

Foreign Key Strategy

The business_id column represents an application-level reference to a Business managed by the Business Service. Because KevaBook follows the Database-per-Service architecture, no database-level foreign key constraint is defined between the Service Catalog Service Database and the Business Service Database. Validation of the business_id is performed through HTTP REST communication with the Business Service.

Relationships

Relationship	Type	Description
Service and Business	Many-to-One	Each Service belongs to one Business.
Service and Staff	Many-to-Many	Implemented through the staff_services junction table.
Service and Bookings	One-to-Many	A Service may have multiple Bookings stored in the Booking Service.

Design Notes

- Every Service belongs to exactly one Business.
- Service pricing is defined independently for each Business.
- The standard Service duration is used during booking validation to calculate appointment end times.
- The status column controls whether a Service is available for booking.
- Service-to-Staff assignments are maintained through the staff_services junction table.
- Other microservices reference Services using the service_id identifier through application-level relationships.

4.3.2 Table Design – staff_services

Purpose

The `staff_services` table defines the assignment of Services to Staff members. It implements the many-to-many relationship between Staff and Services, allowing a Business to specify which Staff members are qualified to perform each Service.

This design provides flexibility for businesses where different Staff members have different skills, certifications, or responsibilities.

Table Structure

Column	Data Type	Constraints	Description
staffServiceId	Long	Primary Key,	Unique identifier for the Staff-Service assignment.
staffId	Long	Not Null	Identifier of the Staff member. This is an application-level reference to the Business Service.
serviceId	Long	Not Null	References the assigned Service within the Service Catalog database.
createdAt	Timestamp	Not Null	Date and time the assignment was created.
updatedAt	Timestamp	Not Null	Date and time the assignment was last updated.

Relationships

Relationship	Type	Description
Staff and Services	Many-to-Many	A Staff member may perform multiple Services.
Service and Staff	Many-to-Many	A Service may be performed by multiple Staff members.

The many-to-many relationship is implemented through the `staff_services` junction table.

Design Notes

- The staff_services table models Staff qualifications rather than booking assignments.
- A Staff member may be assigned to zero, one, or many Services.
- A Service may be assigned to zero, one, or many Staff members.
- Assignment validation is performed by the Service Catalog Service.
- During booking creation, the Booking Service validates that the selected Staff member is authorized to perform the requested Service.
- The unique constraint on (staffId, serviceId) prevents duplicate assignments and maintains data integrity.

4.4 Booking Service Database

Purpose

The Booking Service Database manages all appointment scheduling and booking transactions within the KevaBook platform. It serves as the authoritative source for booking information and staff availability. The database is responsible for validating booking requests, preventing scheduling conflicts, maintaining booking status, and ensuring that appointments comply with business rules and staff availability. As the transactional core of the platform, this database guarantees booking consistency and data integrity across the booking lifecycle.

Tables

Table	Purpose
availability	Stores the recurring working schedules of Staff members.
bookings	Stores appointment bookings created by clients.

Future Enhancements

Additional tables such as booking_history, availability_exceptions, or blocked_time_slots may be introduced in future versions to support auditing, holidays, leave management, and advanced scheduling features.

Database Responsibilities

The Booking Service Database is responsible for:

- Managing Staff availability schedules.
- Creating and updating Bookings.

- Preventing overlapping bookings.
- Validating booking times against Staff availability.
- Managing the booking lifecycle (Pending, Confirmed, Cancelled, Completed).
- Providing booking information to other services through HTTP REST APIs.

The database is **not responsible** for:

- Managing Businesses.
- Managing Staff profiles.
- Managing Services.
- Managing User authentication.
- Sending Notifications.

These responsibilities belong to their respective microservices.

Relationships

The Booking Service Database contains the following internal relationship:

- One Staff Availability record defines a recurring working schedule.
- One Booking references one Availability schedule indirectly through the assigned Staff member and booking time.

Because Staff and Services belong to other microservices, the Booking Service stores only their identifiers.

External References

The Booking Service stores application-level references to entities managed by other services.

Reference	Owned By
business_id	Business Service
staff_id	Business Service
service_id	Service Catalog Service

These references are validated through synchronous HTTP REST communication. No cross-database foreign key constraints are defined.

Design Considerations

The Booking Service Database follows the Database-per-Service architectural pattern, ensuring complete ownership of booking data. To maintain loose coupling between microservices, references to external entities are stored as identifiers rather than database foreign keys. This design allows each microservice to evolve independently while preserving strong transactional consistency within the Booking Service itself. The database is optimized for high-frequency read and write operations, as booking creation, availability checks, and schedule validation represent the most performance-critical workflows within the KevaBook platform.

4.4.1 Table Design – availability

Purpose

The availability table stores the recurring weekly working schedules of Staff members. It defines the days and time periods during which a Staff member is available to accept bookings.

The Booking Service uses this information to validate booking requests and ensure appointments are only scheduled during valid working hours.

Table Structure

Column	Data Type	Constraints	Description
availabilityId	Long	Primary Key	Unique identifier for the availability record.
staffId	Long	Not Null	Identifier of the Staff member. This is an application-level reference to the Business Service.
dayOfWeek	Enum	Not Null	Day of the week
startTime	Time	Not Null	Start of the working period.
endTime	Time	Not Null	End of the working period.

active	Boolean	Not Null	Indicates whether the availability record is active.
createdAt	Timestamp	Not Null	Date and time the record was created.
updatedAt	Timestamp	Not Null	Date and time the record was last updated.

Foreign Key Strategy

The `staff_id` column represents an application-level reference to a Staff member managed by the Business Service. Because KevaBook follows the Database-per-Service architecture, no database-level foreign key constraint exists between the Booking Service Database and the Business Service Database. Validation of the `staff_id` is performed through synchronous HTTP REST communication.

Relationships

Relationship	Type	Description
Availability and Staff	Many-to-One (Application-level)	Each availability record belongs to one Staff member.

A Staff member may have multiple availability records representing different working days or shifts.

Design Notes

- Availability is defined as a recurring weekly schedule rather than individual calendar dates.
- A Staff member may have multiple availability records for different days or split shifts within the same day.
- Multiple availability records for the same day support working patterns such as 09:00–12:00 and 13:00–17:00.
- The Booking Service validates every booking against the Staff member's active availability records before confirming the appointment.
- Temporary schedule changes, vacations, holidays, and other exceptions are not stored in this table and will be supported through dedicated scheduling tables in future versions.

4.4.2 Table Design – bookings

Purpose

The bookings table stores appointment reservations created by clients. Each Booking represents a scheduled Service provided by a Staff member for a specific Business at a defined date and time. The table serves as the primary transactional entity within the KevaBook platform and is responsible for maintaining booking lifecycle information while supporting scheduling validation and business rule enforcement.

Table Structure

Column	Data Type	Constraints	Description
booking_id	Long	Primary Key	Unique identifier for the Booking.
business_id	Long	Not Null	Identifier of the Business. Application-level reference to the Business Service.
staff_id	Long	Not Null	Identifier of the assigned Staff member. Application-level reference to the Business Service.
service_id	Long	Not Null	Identifier of the booked Service. Application-level reference to the Service Catalog Service.
client_name	String	Not Null	Full name of the client making the booking.
client_email	String	Not Null	Client's email address.
client_phone	String	Null	Client's contact phone number.
booking_date	Date	Not Null	Date of the appointment.
start_time	Time	Not Null	Scheduled appointment start time.
end_time	Time	Not Null	Scheduled appointment end time.
status	Enum	Not Null	Booking status (PENDING, CONFIRMED, CANCELLED, COMPLETED).

notes	String	Null	Optional notes provided by the client or Business.
created_at	Timestamp	Not Null	Date and time the Booking was created.
updated_at	Timestamp	Not Null	Date and time the Booking was last updated.

Foreign Key Strategy

The following columns are application-level references:

Column	Owned By
business_id	Business Service
staff_id	Business Service
service_id	Service Catalog Service

Because KevaBook follows the Database-per-Service architecture, no database-level foreign key constraints are created for these references. Validation is performed through synchronous HTTP REST communication before a Booking is created.

Relationships

Relationship	Type	Description
Booking and Business	Many-to-One	Each Booking belongs to one Business.
Booking and Staff	Many-to-One	Each Booking is assigned to one Staff member.
Booking and Service	Many-to-One	Each Booking is created for one Service.

Design Notes

- Each Booking belongs to exactly one Business, one Staff member, and one Service.

- Client information is stored within the Booking to preserve historical accuracy, even if client details change in the future.
- The Booking Service validates that:
 - the Staff member exists,
 - the Service exists,
 - the Staff member is qualified to perform the Service,
 - the requested time falls within the Staff member's availability,
 - no overlapping Booking exists for the assigned Staff member.
- Booking status transitions are managed by the Booking Service according to the defined business rules.
- Audit fields support operational monitoring and future reporting requirements.

4.5 Notification Service Database

Purpose

The Notification Service Database manages all outbound notifications generated by the KevaBook platform. It stores notification records, tracks delivery status, and maintains a history of email and sms messages sent to clients and Business Owners. The Notification Service consumes events published through RabbitMQ and is responsible for processing, delivering, and monitoring notifications independently of the services that generate those events.

Database Responsibilities

The Notification Service Database is responsible for:

- Persisting notification requests received from RabbitMQ.
- Tracking email and sms delivery status.
- Recording notification processing timestamps.
- Supporting retry mechanisms for failed deliveries.
- Maintaining notification history for auditing and troubleshooting.

The database is **not responsible** for:

- Creating Bookings.
- Managing Businesses.
- Managing Services.
- Managing Users.
- Managing Staff.

These responsibilities belong to their respective microservices.

Relationships

The Notification Service Database does not maintain direct relational dependencies with other service databases. Instead, notification records store application-level references to related domain entities.

External References

The Notification Service stores identifier references received from published domain events.

Examples include:

Reference	Owned By
bookingId	Booking Service
businessId	Business Service

These references are maintained for traceability only. No cross-database foreign key constraints are defined.

Design Considerations

The Notification Service follows the Database-per-Service architectural pattern. Notifications are persisted independently to ensure reliable message processing, support retry operations, and provide an audit trail of all outbound communications. The service is designed to be event-driven and remains loosely coupled from the rest of the platform through RabbitMQ.

4.5.1 Table Design – notifications

Purpose

The notifications table stores all outbound notification requests processed by the Notification Service. Each record represents a notification generated in response to a business event, such as a booking confirmation, cancellation, reminder, or status update.

The table maintains notification content, delivery status, retry information, and processing history to ensure reliable message delivery and support auditing.

Table Structure

Column	Data Type	Constraints	Description
notificationId	Long	Primary Key	Unique identifier for the notification.
bookingId	Long	Null	Identifier of the related Booking.
businessId	Long	Null	Identifier of the related Business.
notificationType	String	Not Null	Notification channel (EMAIL, SMS).
recipient	String	Not Null	Recipient email address or phone number.
subject	String	Null	Notification subject. Used primarily for email notifications.
message	String	Not Null	Notification content sent to the recipient.
status	Enum	Not Null	Current delivery status (PENDING, SENT, FAILED).
retryCount	Integer	Not Null	Number of delivery retry attempts.
failureReason	String	NULL	Description of the failure when delivery is unsuccessful.

sentAt	Timestamp	Null	Date and time the notification was successfully sent.
createdAt	Timestamp	Not Null	Date and time the notification record was created.
updatedAt	Timestamp	Not Null	Date and time the notification record was last updated.

Foreign Key Strategy

The following columns are application-level references:

Column	Owned By
bookingId	Booking Service
businessId	Business Service

Because KevaBook follows the Database-per-Service architecture, no database-level foreign key constraints are defined. These identifiers are stored solely for traceability and correlation with domain events.

Relationships

The Notification Service Database does not maintain database relationships with other microservices. Notification records are linked to external entities through application-level identifier references.

Design Notes

- Notification requests are created after consuming domain events from RabbitMQ.
- Each notification is processed independently to prevent failures from affecting other services.
- The status field tracks the notification lifecycle from creation to successful delivery or failure.
- Failed notifications may be retried according to the configured retry policy.
- Email and SMS notifications share the same table structure to simplify processing and reporting.
- The design can be extended to support additional notification channels, such as push notifications or in-app messages, without requiring structural changes to the database.